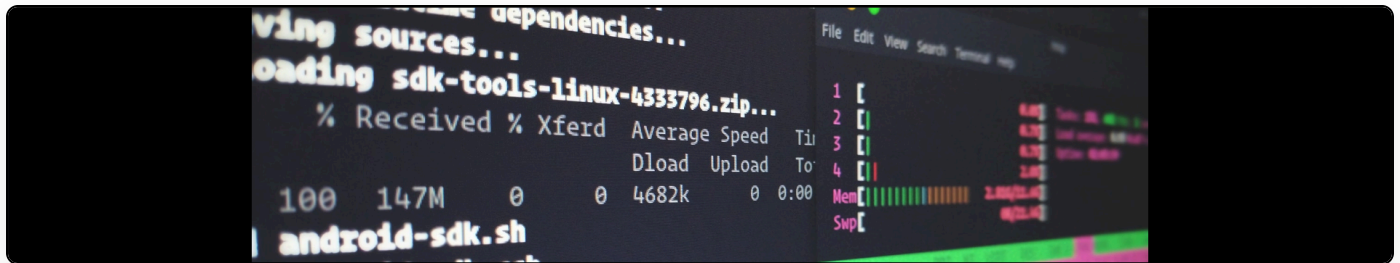


Writing a plugin for Open Journal Systems: Part 2

Cite as: Martin Paul Eve, "Writing a plugin for Open Journal Systems: Part 2", *eve.gd: Martin Paul Eve*, September 25, 2011, <https://doi.org/10.59348/agcg2-btm24>.



The quest to build a system that allows [publishing in PDF and XHTML](#) from a single XML galley within OJS [continues](#) and I've made quite substantial progress.

As before, the code for this article is available on [my GitHub](#).

Instead of working from the very basic stub, I instead used the existing xmlGalley template, reading and tracing through the code to work out what it does. The easiest way to understand the plugin, if you don't have a working installation running under Eclipse, is to whack some error_log statements inside each method. The way this plugin works is that, on the call to display a galley file, it checks if there is an associated entry in the xml_galleys table. If there is, it works off that instead. It then asks if the cache has an already extant entry for the XSLT transform. If not, it performs the transform.

My plan to rework this, and get around the bug, was to first modify the hook method when a galley is inserted. In this case, if the galley inserted is an XML file, I copy the entry to a new file, named "PDF". Poor form, perhaps, but I'm using the name "PDF" to determine that the galley should be transformed to a PDF file.

The way that I performed this copy was to modify the insertXMLGalleys function in ArticleXMLDao.inc.php:

```

/**
 * Insert XML-derived galleys into article_xml_galleys
 */
function insertXMLGalleys($hookName, $args) {

    $galley =& $args[0];
    $galleyId =& $args[1];

    // If the galley is an XML file, then insert rows in the
    article_xml_galleys table
    if ($galley->getLabel() == "XML") {

        // create an XHTML galley
        $this->update(
            'INSERT INTO article_xml_galleys
              (galley_id, article_id, label, galley_type)
              VALUES
              (?, ?, ?, ?)',
            array(
                $galleyId,
                $galley->getArticleId(),
                'XHTML',
                'application/xhtml+xml'
            )
        );

        // if we have enabled XML-PDF galley generation (plugin
        setting)
        // and are using the built-in NLM stylesheet, append a PDF
        galley as well
        // this will insert a second corresponding entry into
        article_galleys first in order
        // to circumvent bug #5152 by only ever having one galley per
        file

        // insert the PDF/XML galley
        $journal =& Request::getJournal();
        $xmlGalleyPlugin =& PluginRegistry::getPlugin('generic', $this-
        >parentPluginName);

        if ($xmlGalleyPlugin->getSetting($journal->getId(), 'nlmPDF')
        == 1 &&
            $xmlGalleyPlugin->getSetting($journal->getId(),
            'XSLstylesheet') == 'NLM' ) {

```

```

        // instantiate a new galley file
        $ArticleGalley = new ArticleXMLGalley('meXml');

        $ArticleGalley->setArticleId($galley->getArticleId());
        $ArticleGalley->setLabel('PDF');
        $ArticleGalley->setLocale(Locale::getLocale());
        $ArticleGalley->setFileId($galley->getFileId());
        $ArticleGalley->setFileType('application/pdf');
        $ArticleGalley->setType('public');

        // before the insert, we have to clear the hooks, or we
get an infinite loop

HookRegistry::clear('ArticleGalleyDAO::insertNewGalley');

        // insert the galley
        $galleyDao =& DAORegistry::getDAO('ArticleGalleyDAO');
        $galleyDao->insertGalley($ArticleGalley);

        // re-register the hook

HookRegistry::register('ArticleGalleyDAO::insertNewGalley', array(&$this,
'insertXMLGalleys') );

        // create a PDF galley
        $this->update(
            'INSERT INTO article_xml_galleys
            (galley_id, article_id, label,
galley_type)
            VALUES
            (?, ?, ?, ?)',
            array(
                $ArticleGalley->getId(),
                $galley->getArticleId(),
                'PDF',
                'application/pdf'
            )
        );

    }
    return true;
}
return false; }

```

To enable the name based selection, I put the following if block inside meXml.inc.php's `_returnXMLGalleyFromArticleGalley`:

```
// override file type based on name
if($galley->getLabel() == 'PDF')
{
    $articleXMLGalley->setFileType('application/pdf');
} else {
    $articleXMLGalley->setFileType($galley->getFileType());
}
```

All was looking good and, sure enough, when I fired up the site, uploaded an XML file, it copied the new PDF version. There is, however, a problem. The FOP transform that was supposed to take place afterwards was utterly failing. Eventually, I downloaded the latest FO and XHTML transforms from [the NLM site](#) and modified the code to use these... except, they're XSLT 2.0. This means that a new parser was needed; PHP uses libXSLT which doesn't support, and doesn't intend to support, XSLT 2.0.

For this, then, I had to use saxon. This adds a dependency upon java and libsaxon, but this seems unavoidable. My additional saxon transform statement looks like this:

```

if ( $xsltType == "saxon" ) {
    // PDF transform using java, saxon and XSLT 2.0

    // TODO: this needs to be loaded from a setting
    $xsltType = '/usr/bin/saxonb-xslt -ext:on %xml %xsl';

    // parse the external command to check for %xsl and %xml
parameter substitution
    if ( strpos($xsltType, '%xsl') === false ) return false;

    // perform %xsl and %xml replacements for fully-qualified shell
command
    $xsltCommand = str_replace(array('%xsl', '%xml'),
array($xslFile, $xmlFile), $xsltType);

    // check for safe mode and escape the shell command
    if( !ini_get('safe_mode') ) $xsltCommand =
escapeshellcmd($xsltCommand);

    // run the shell command and get the results
    exec($xsltCommand . ' 2>&1', $contents, $status);

    // if there is an error, spit out the shell results to aid
debugging
    if ($status != false) {
        if ($contents != '') {
            echo implode("\n", $contents);
            return true;
        } else return false;
    }

    return implode("\n", $contents);
}

```

I thought this looked good to go... but no!

The NLM XLST-FO files create incompatible results with FOP in the references list. For instance, the following XML fragment:

```
<ref id="B3">
<label>3</label>
<citation citation-type="journal">
<person-group person-group-type="author">
<name>
<surname>Ruru</surname>
<given-names>Li</given-names>
</name>
</person-group>
<article-title>Shakespeare on the Chinese Stage in the 1990s</article-title>
<source>Shakespeare Quarterly</source>
<year>1999</year>
<issue>3</issue>
<volume>50</volume>
<fpage>355</fpage>
<lpag>367</lpag>
<issn pub-type="ppub">00373222</issn>
<pub-id pub-id-type="doi">10.2307/2902363</pub-id>
<uri>
http://www.jstor.org/stable/2902363?origin=crossref
</uri>
</citation>
</ref>
```

Looks good. However, after transform, it generates FO which chokes FOP. To fix this, I had to make some modifications to the NLM transform styles which are in the Git repo. These changes included removing a fo:wrapper from the ref list and changing the make-external-link match to read thus:

```

<xsl:template name="make-external-link">

  <xsl:param name="href">
    <xsl:choose>
      <xsl:when test="normalize-space()">
        <xsl:apply-templates/>
      </xsl:when>
      <xsl:otherwise>
        <xsl:value-of select="@contents"/>
      </xsl:otherwise>
    </xsl:choose>
  </xsl:param>

  <xsl:param name="contents">

    <xsl:choose>

      <xsl:when test="normalize-space()">

        <xsl:apply-templates/>

      </xsl:when>

      <xsl:otherwise>

        <xsl:value-of select="@xlink:href"/>

      </xsl:otherwise>

    </xsl:choose>

  </xsl:param>

  <fo:basic-link external-destination="{normalize-space($href)}"

    show-destination="new" xsl:use-attribute-sets="link">

    <xsl:copy-of select="$contents"/>

  </fo:basic-link>

</xsl:template>

```

From this, OJS can now generate rudimentary PDFs and XHTML output from the same uploaded file. It's very basic, somewhat unstable and has some hard-coded features that must be shipped out to userland options. That said, it's a good start!